

Livrable : Application Streamlit de Présentation des Résultats

Description

Une application **Streamlit** a été développée pour consommer l'API FastAPI déployée sur Azure. Cette application permet de :

1. **Charger une image** depuis l'interface.
2. **Envoyer l'image à l'API FastAPI** pour obtenir le masque segmenté.
3. **Afficher les résultats** : présenter l'image d'entrée ainsi que le masque prédit en nuances de gris, sans application de color mapping (une décision à confirmer avec Laura si nécessaire).
4. **Télécharger le masque prédit** pour une analyse ou un usage ultérieur.

Ce livrable illustre le fonctionnement de la chaîne complète du modèle de segmentation.

Détails des Livrables

- **Lien Azure de l'Application Streamlit déployée** :
<http://streamlit-p8.westeurope.azurecontainer.io:8501>
- **Code source sur GitHub** :
Le code source de l'application est disponible ici :
https://github.com/preudh/OC_IA_P8_VoitureAutonome

Structure de l'Application

L'application est organisée comme suit :

```
app/
├── Dockerfile           # Fichier pour la création de l'image Docker
├── .dockerignore        # Liste des fichiers/dossiers ignorés par Docker
├── app.py               # Script principal pour Streamlit
└── requirements.txt     # Liste des dépendances nécessaires
```

Fonctionnement de l'Application Streamlit

L'application **Streamlit** interagit avec l'API FastAPI de la manière suivante :

1. L'utilisateur charge une image via l'interface de l'application.
2. L'application envoie cette image à l'API FastAPI déployée sur Azure via une requête POST au point de terminaison `/predict`.
 - URL de l'API : <http://fastapi-p8.westeurope.azurecontainer.io:8000/predict>
3. L'API retourne le masque prédit, qui est affiché dans l'application aux côtés de l'image originale.
4. L'utilisateur peut ensuite télécharger le masque généré au format PNG.

Points Techniques Importants

- **Requêtes HTTP** : L'interaction entre Streamlit et FastAPI repose sur le module `requests` pour envoyer les images et recevoir les masques.
- **Docker** : L'application a été conteneurisée pour faciliter le déploiement et assurer la portabilité.
- **Déploiement sur Azure** : L'application est hébergée sur Azure dans un conteneur Docker.

Exemple de Fonctionnalité (Extrait de Code)

```
# Soumission de l'image
if uploaded_file:
    image = Image.open(uploaded_file)
    response = requests.post(API_URL, files={"file": img_bytes})

    if response.status_code == 200:
        # Affichage des résultats
        st.image(image, caption="Image d'entrée")
        st.image(predicted_mask, caption="Masque prédit")
        st.download_button(
            label="Télécharger le masque",
            data=response.content,
            file_name="predicted_mask.png",
            mime="image/png"
        )
```

Axe d'Amélioration

Une **approche CI/CD** pourrait être mise en œuvre pour automatiser et fiabiliser le déploiement :

1. CI/CD avec GitHub Actions :

- Tester et construire automatiquement l'image Docker (pipeline CI).
- Pousser l'image vers un registre tel qu'Azure Container Registry (ACR) pour le déploiement (pipeline CD).

2. Utilisation de plusieurs fichiers Docker Compose :

- Trois fichiers distincts pourraient être utilisés pour gérer les environnements :
 - `docker-compose.ci.yml` : pour les tests CI.
 - `docker-compose.dev.yml` : pour le développement local.
 - `docker-compose.prod.yml` : pour le déploiement en production.

3. Automatisation avec GitHub Actions et Docker Compose :

- Le fichier `.yml` de GitHub Actions pourrait orchestrer le pipeline CI/CD et utiliser les fichiers Docker Compose pour gérer chaque étape.

Cette approche garantirait une meilleure reproductibilité, une réduction des efforts manuels, et une intégration fluide dans une logique **Mlops**.

Bien que ce processus n'ait pas été entièrement automatisé ici, l'architecture actuelle reste une base solide pour une future mise en place de cette automatisation.